



Figure 12: Multi-Class Multi-Label Classification: Classification Accuracy (left), Satisfiability level (middle), and Querying of Constraints (right).

Single Digits Addition: Consider the predicate $\text{addition}(X, Y, N)$, where X and Y are images of digits (the MNIST data set will be used), and N is a natural number corresponding to the sum of these digits. This predicate should return an estimate of the validity of the addition. For instance, $\text{addition}(\text{3}, \text{8}, 11)$ is a valid addition; $\text{addition}(\text{3}, \text{8}, 5)$ is not.

Multi Digits Addition: The experiment is extended to numbers with more than one digit. Consider the predicate $\text{addition}([X_1, X_2], [Y_1, Y_2], N)$. $[X_1, X_2]$ and $[Y_1, Y_2]$ are lists of images of digits, representing two multi-digit numbers; N is a natural number corresponding to the sum of the two multi-digit numbers. For instance, $\text{addition}([\text{3}, \text{8}], [\text{9}, \text{2}], 130)$ is a valid addition; $\text{addition}([\text{3}, \text{8}], [\text{9}, \text{2}], 26)$ is not.

A natural neurosymbolic approach is to seek to learn a single-digit classifier and benefit from knowledge readily available about the properties of addition in this case. For instance, suppose that a predicate $\text{digit}(x, d)$ gives the likelihood of an image x being of digit d . A definition for $\text{addition}(\text{3}, \text{8}, 11)$ in LTN is:

$$\exists d_1, d_2 : d_1 + d_2 = 11 \ (\text{digit}(\text{3}, d_1) \wedge \text{digit}(\text{8}, d_2))$$

In [41], the above task is made more complicated by not providing labels for the single-digit images during training. Instead, training takes place on pairs of images with labels made available for the result only, that is, the sum of the individual labels. The single-digit classifier is not explicitly trained by itself; its output is a piece of latent information that is used by the logic. However, this does not pose a problem for end-to-end neurosymbolic systems such as LTN or DeepProbLog for which the gradients can propagate through the logical structures.

We start by illustrating a LTN theory that can be used to learn the predicate digit . The specification of the theory below is for the single digit addition example, although it can be extended easily to the multiple digits case.

Domains:

images, denoting the MNIST digit images,
 results, denoting the integers that label the results of the additions,
 digits, denoting the digits from 0 to 9.

Variables:

x, y , ranging over the MNIST images in the data,
 n for the labels, i.e. the result of each addition,
 d_1, d_2 ranging over digits.

$\mathbf{D}(x) = \mathbf{D}(y) = \text{images},$
 $\mathbf{D}(n) = \text{results},$
 $\mathbf{D}(d_1) = \mathbf{D}(d_2) = \text{digits}.$

Predicates:

$\text{digit}(x, d)$ for the single digit classifier, where d is a term denoting a digit constant or a digit variable. The classifier should return the probability of an image x being of digit d .
 $\mathbf{D}_{\text{in}}(\text{digit}) = \text{images, digits}.$

Axioms:

Single Digit Addition:

$$\begin{aligned}
&\forall \text{Diag}(x, y, n) \\
&(\exists d_1, d_2 : d_1 + d_2 = n \\
&(\text{digit}(x, d_1) \wedge \text{digit}(y, d_2)))
\end{aligned} \tag{39}$$

Multiple Digit Addition:

$$\begin{aligned}
&\forall \text{Diag}(x_1, x_2, y_1, y_2, n) \\
&(\exists d_1, d_2, d_3, d_4 : 10d_1 + d_2 + 10d_3 + d_4 = n \\
&(\text{digit}(x_1, d_1) \wedge \text{digit}(x_2, d_2) \wedge \text{digit}(y_1, d_3) \wedge \text{digit}(y_2, d_4)))
\end{aligned} \tag{40}$$

Notice the use of Diag : when grounding x, y, n with three sequences of values, the i -th examples of each variable are matching. That is, $(\mathcal{G}(x)_i, \mathcal{G}(y)_i, \mathcal{G}(n)_i)$ is a tuple from our dataset of valid additions. Using the diagonal quantification, LTN aggregates pairs of images and their corresponding result, rather than any combination of images and results.

Notice also the guarded quantification: by quantifying only on the latent "digit labels" (i.e. $d_1, d_2, \dots$) that can add up to the result label (n , given in the dataset), we incorporate symbolic information into the system. For example, in (39), if $n = 3$, the only valid tuples (d_1, d_2) are $(0, 3), (3, 0), (1, 2), (2, 1)$. Gradients will only backpropagate to these values.

Grounding:

$\mathcal{G}(\text{images}) = [0, 1]^{28 \times 28 \times 1}$. The MNIST data set has images of 28 by 28 pixels. The images are grayscale and have just one channel. The RGB pixel values from 0 to 255 of the MNIST data set are converted to the range $[0, 1]$.

$\mathcal{G}(\text{results}) = \mathbb{N}$.

$\mathcal{G}(\text{digits}) = \{0, 1, \dots, 9\}$.

$\mathcal{G}(x) \in [0, 1]^{m \times 28 \times 28 \times 1}, \mathcal{G}(y) \in [0, 1]^{m \times 28 \times 28 \times 1}, \mathcal{G}(n) \in \mathbb{N}^m$.²⁴

$\mathcal{G}(d_1) = \mathcal{G}(d_2) = \langle 0, 1, \dots, 9 \rangle$.

$\mathcal{G}(\text{digit} \mid \theta) : x, d \mapsto \text{onehot}(d)^\top \cdot \text{softmax}(\text{CNN}_\theta(x))$, where CNN is a Convolutional Neural Network with 10 output neurons for each class. Notice that, in contrast with the previous examples, d is an integer label; $\text{onehot}(d)$ converts it into a one-hot label.

²⁴Notice the use of the same number m of examples for each of these variables as they are supposed to match one-to-one due to the use of Diag .

Learning:

The computational graph of Figure 13 shows the objective function for the satisfiability of the knowledge base. A stable product configuration is used with hyper-parameter $p = 2$ of the operator A_{pME} for universal quantification ($\forall$). Let $p_{\exists}$ denote the exponent hyper-parameter used in the generalized mean A_{pM} for existential quantification ($\exists$). Three scenarios are investigated and compared in the Multiple Digit experiment (Figure 15):

1. $p_{\exists} = 1$ throughout the entire experiment,
2. $p_{\exists} = 2$ throughout the entire experiment, or
3. $p_{\exists}$ follows a schedule, changing from $p = 1$ to $p = 6$ gradually with the number of training epochs.

In the Single Digit experiment, only the last scenario above (schedule) is investigated (Figure 14).

We train to maximize satisfiability by using batches of 32 examples of image pairs, labeled by the result of their addition. As done in [41], the experimental results vary the number of examples in the training set to emphasize the generalization abilities of a neurosymbolic approach. Accuracy is measured by predicting the digit values using the predicate digit and reporting the ratio of examples for which the addition is correct. A comparison is made with the same baseline method used in [41]: given a pair of MNIST images, a non-pre-trained CNN outputs embeddings for each image (Siamese neural network). The embeddings are provided as input to dense layers that classify the addition into one of the 19 (respectively, 199) possible results of the Single Digit Addition (respectively, Multiple Digit Addition) experiments. The baseline is trained using a cross-entropy loss between the labels and the predictions. As expected, such a standard deep learning approach struggles with the task without the provision of symbolic meaning about intermediate parts of the problem.

Experimentally, we find that the optimizer for the neurosymbolic system gets stuck in a local optimum at the initialization in about 1 out of 5 runs. We, therefore, present the results on an average of the 10 best outcomes out of 15 runs of each algorithm (that is, for the baseline as well). The examples of digit pairs selected from the full MNIST data set are randomized at each run.

Figure 15 shows that the use of $p_{\exists} = 2$ from the start produces poor results. A higher value for $p_{\exists}$ in A_{pM} weighs up the instances with a higher truth-value (see also Appendix C for a discussion). Starting already with a high value for $p_{\exists}$, the classes with a higher initial truth-value for a given example will have higher gradients and be prioritized for training, which does not make practical sense when randomly initializing the predicates. Increasing $p_{\exists}$ by following a schedule is the most promising approach. In this particular example, $p_{\exists} = 1$ is also shown to be adequate purely from a learning perspective. However, $p_{\exists} = 1$ implements a simple average which does not account for the meaning of $\exists$ well; the resulting satisfaction value is not meaningful within a reasoning perspective.

Table 1 shows that the training and test times of LTN are of the same order of magnitude as those of the CNN baselines. Table 2 shows that LTN reaches similar accuracy as that reported by DeepProbLog.